

Project Report

Implement Client Script & UI Policy (Incident)

Platform	ServiceNow
Team ID	SWTID-2026-8713
Date	03-10-2026
Duration	5 hours 15 minutes (7 milestones)

Contents

1. Introduction	3
1.1 Project Overview	3
1.2 Purpose	3
2. Ideation Phase	3
2.1 Problem Statement	3
2.2 Empathy Map Canvas	3
2.3 Brainstorming	4
3. Requirement Analysis	4
3.1 Customer Journey Map	4
3.2 Solution Requirement	4
3.3 Data Flow Diagram	5
3.4 Technology Stack	5
4. Project Design	6
4.1 Problem Solution Fit	6
4.2 Proposed Solution	6
4.3 Solution Architecture	6
5. Project Planning & Scheduling	6
5.1 Project Planning	6
6. Functional and Performance Testing	7
6.1 Performance Testing	7
7. Results	8
7.1 Output Screenshots	8
8. Advantages & Disadvantages	9
9. Conclusion	9
10. Future Scope	9
11. Appendix	9
Source Code	9
Dataset Link	10
GitHub & Project Demo Link	10

1. Introduction

1.1 Project Overview

This project implements Client Scripts and UI Policies on the ServiceNow Incident form. UI Policies make selected fields mandatory, read-only or visible when defined conditions are met, and Client Scripts (onLoad, onChange and onSubmit) handle dynamic interaction and validation in the browser. The form reacts immediately to what the user does, without having to be submitted, so Incident records are more complete and consistent.

1.2 Purpose

The purpose is to show how UI Policies and Client Scripts work together to guide users on the Incident form: required information is captured, protected fields cannot be edited when they should not be, and unnecessary fields are hidden. The result is better data quality, less rework and a clearer form for service desk agents.

2. Ideation Phase

2.1 Problem Statement

The Incident form does not automatically enforce the required field behavior for every condition. Users may leave important fields incomplete, edit fields that should be protected, or see fields that are not needed. The administrator needs to configure fields to become mandatory, read-only or visible when defined conditions are met, and the developer needs rules that are clear, tested and free of conflicts.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A ServiceNow administrator responsible for the Incident form	Configure Incident fields to become mandatory, read-only, or visible when defined conditions are met	Users may leave important fields incomplete or edit fields that should be protected	The form does not automatically enforce the required field behavior for every condition	Concerned about inconsistent Incident data and manual checking
PS-2	A ServiceNow developer who configures Client Scripts and UI Policies	Implement dynamic client-side behavior for Incident fields	Rules can become inconsistent or conflict if they are not clearly defined and tested	Different Incident states and field values require different form behaviors	Uncertain about whether the form will respond correctly to every user action
PS-3	A service desk agent / end user	Create and update Incident records using a clear and consistent form	I may not know which fields are required, editable, or relevant at a particular time	Field behavior is not dynamically controlled according to Incident conditions	Confused and frustrated

2.2 Empathy Map Canvas

User: ServiceNow administrator / developer.

SEES	SAYS AND DOES
- Incident fields that may be left incomplete - Fields that remain editable when they should be protected - Unnecessary fields displayed for every condition - Inconsistent data-entry practices between Incident records	- Creates or updates Incident records - Changes fields such as State, Category, Subcategory and Assignment Group - Expects fields to become mandatory, read-only or visible when conditions are met - Uses the form without needing to submit it to trigger field behavior

PAIN	GAIN
• Missing required Incident information • Users can edit fields that should be protected • Too many irrelevant fields make the form confusing • Manual validation increases the chance of inconsistent data	• Required information is captured consistently • Restricted fields become read-only when appropriate • Relevant fields are shown based on Incident conditions • Better user experience and more accurate Incident records

2.3 Brainstorming

The team considered several ways to control the Incident form and compared them.

Idea	Pros	Cons	Decision
Train agents and check data manually	No configuration needed	Errors still occur; wrong data is found only after saving	Rejected
UI Policies for mandatory, read-only and visible fields	Easy to build without code; reverses automatically	Cannot handle complex logic alone	Selected
Client Scripts (onLoad, onChange, onSubmit)	Form reacts instantly; supports validation and messages	Need testing; errors are harder to trace	Selected
Server-side validation only	Works for every channel	User gets feedback only after the save is attempted	Not selected for this project

Ideas were grouped into Mandatory Control, Read-only Control, Visibility Control, Dynamic Interaction and Validation & Testing, and prioritised so that field control comes first, followed by Client Script interaction and testing.

3. Requirement Analysis

3.1 Customer Journey Map

Stage	User action	Experience	Outcome
Entice	Agents complain that the form is confusing; administrator finds missing close notes	Awareness of the problem	Need for form rules is clear
Enter	Opens the Incident form, Client Scripts and UI Policies modules; identifies fields to control	UI Policies are easy to build; unsure when to use a Client Script	Fields and conditions listed
Engage	Adds UI Policy actions and writes onLoad, onChange, onSubmit scripts using g_form	The form reacts straight away when a field changes	Rules configured
Exit	Verifies each rule for every State, Category and Priority; checks the console	Confidence that mandatory and read-only rules work	Form cannot be saved with invalid data
Extend	Documents the outcome; reuses the patterns; updates rules when fields change	Fewer corrections and follow-up questions	Cleaner, more complete records

3.2 Solution Requirement

ID	Requirement	Type
FR-1	UI Policies with conditions: mandatory, visible and read-only actions, reversed when the condition no longer applies	Functional
FR-2	onLoad Client Script that sets the initial form state and shows guidance	Functional

ID	Requirement	Type
FR-3	onChange Client Scripts on Category, State, Impact and Urgency that update dependent fields	Functional
FR-4	onSubmit Client Script that validates data and stops an invalid save	Functional
FR-5	Field messages, form info messages and alerts for invalid entries	Functional
FR-6	Test each rule for every State and Category and check the browser console	Functional
FR-7	Activate, deactivate and update policies and scripts	Functional
NFR-1	Usability – only relevant fields are shown, mandatory fields are clearly marked	Non-functional
NFR-2	Security – client rules guide users; access is enforced on the server by roles and ACLs	Security
NFR-3	Reliability – the same field values always give the same behavior	Non-functional
NFR-4	Performance – rules run in the browser and avoid synchronous server calls	Non-functional
NFR-5	Availability – rules are delivered with the form from the ServiceNow cloud instance	Non-functional
NFR-6	Scalability – patterns can be reused for other tables; rules move with update sets	Non-functional

3.3 Data Flow Diagram

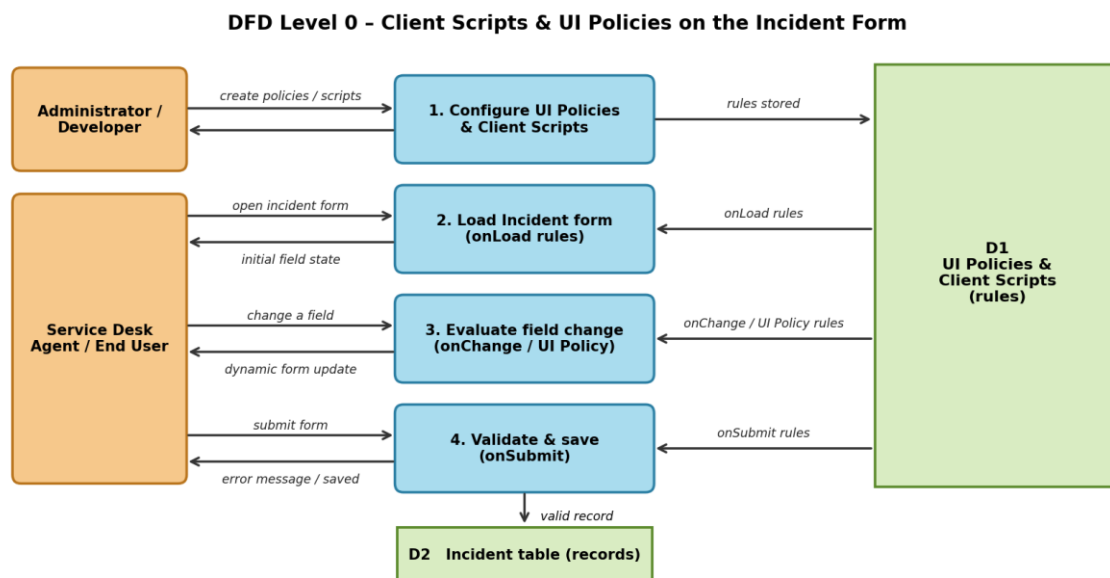


Figure: DFD Level 0 – Client Scripts & UI Policies on the Incident form

3.4 Technology Stack

Layer	Technology
Platform	ServiceNow cloud instance
User interface	ServiceNow Platform UI – Incident form (HTML, CSS, JavaScript in the browser)
Form rules	UI Policies (sys_ui_policy) with Execute if true / false scripts
Scripting	Client-side JavaScript – Client Scripts (sys_script_client) using g_form and g_user
Data store	Incident table (incident)
Security	Roles and ACLs (server-side); Impersonate User for testing
Testing	Incident form, browser developer console and a timer

4. Project Design

4.1 Problem Solution Fit

Problem	Solution fit
Important fields are left incomplete	UI Policy makes Close code, Close notes and Short description mandatory when conditions are met
Users edit fields that should be protected	UI Policy makes fields read-only for closed incidents
Unnecessary fields shown for every condition	UI Policy shows or hides fields such as On hold reason by State and Category
Bad data is found only after the incident is saved	onChange and onSubmit Client Scripts react instantly and block invalid saves with a clear message

4.2 Proposed Solution

Use a combined Client Script and UI Policy approach on the Incident table. UI Policies control mandatory, read-only and visible behavior when defined conditions are met, while Client Scripts handle dynamic client-side interactions that need additional logic.

Rule type	Runs	Used for
UI Policy	When its condition is true / false	Mandatory, read-only and visible fields
Client Script – onLoad	When the form opens	Initial state and guidance messages
Client Script – onChange	When a field value changes	Updating dependent fields and field messages
Client Script – onSubmit	When the user saves	Validating data and stopping an invalid save

4.3 Solution Architecture

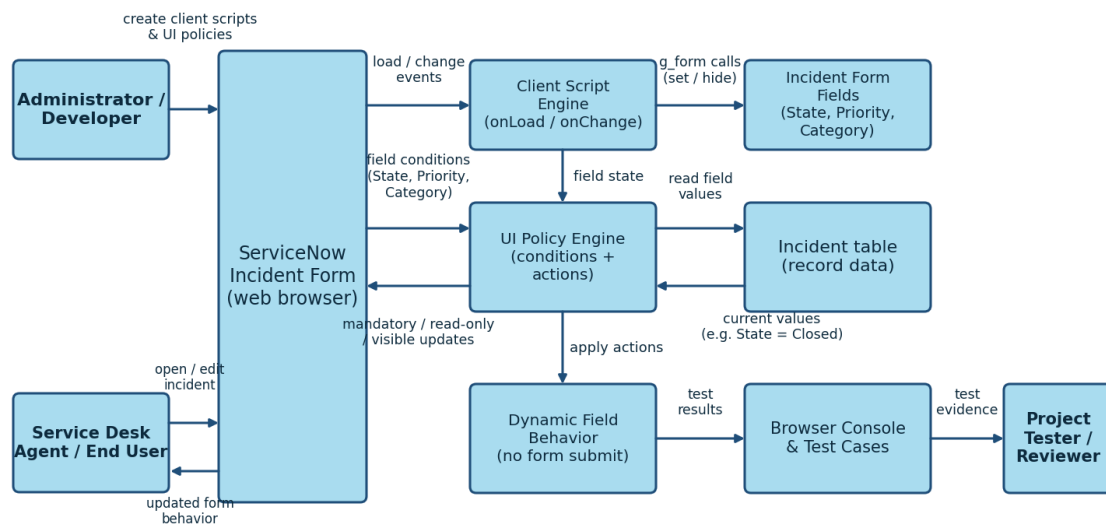


Figure: Solution architecture – Client Scripts and UI Policies on ServiceNow

5. Project Planning & Scheduling

5.1 Project Planning

The project is planned as seven milestones (5 hours 15 minutes in total) and three sprints of 10 days each, with 30 story points.

Milestone	Activity	Owner	Duration	Status
Milestone-1	Identify Incident fields and conditions	Vishal	30 min	To do
Milestone-2	Configure UI Policy to make selected fields mandatory	Thabeshwin	45 min	To do
Milestone-3	Configure UI Policy to make selected fields read-only	Thabeshwin	45 min	To do
Milestone-4	Configure UI Policy to show/hide selected fields	Supriya	45 min	To do
Milestone-5	Implement dynamic Incident field behavior	Supriya	1 hr	To do
Milestone-6	Test conditions, field behavior and user actions	Sakthivel S	1 hr	To do
Milestone-7	Document project outcome and verify consistent data entry	Sakthivel S	30 min	To do
Total			5 hrs 15 min	

Sprint	Start	End (planned)	Story points
Sprint-1	05 Oct 2026	16 Oct 2026	10
Sprint-2	19 Oct 2026	30 Oct 2026	12
Sprint-3	02 Nov 2026	13 Nov 2026	8

6. Functional and Performance Testing

6.1 Performance Testing

10 functional and 3 performance test cases are defined. Performance acceptance criteria: the form loads in under 3 seconds and fields react in under 1 second. Actual results and status are recorded by the tester while executing the tests.

Test ID	Scenario	Expected result	Status
FT-01	Short description mandatory	Short description is marked mandatory and the incident is not saved until it is filled.	Pending
FT-02	Close code / Close notes mandatory on Resolved	Close code and Close notes become mandatory immediately and the save is blocked until both are filled.	Pending
FT-03	Mandatory rule reverses	Close code and Close notes are no longer mandatory; the form can be saved.	Pending
FT-04	Show / hide On hold reason	On hold reason appears for On Hold and is hidden for other States without reloading the form.	Pending
FT-05	Read-only fields on closed incident	The controlled fields are read-only and cannot be edited.	Pending
FT-06	onLoad initial state and guidance	The form opens with the correct default state and shows the guidance message.	Pending
FT-07	onChange updates dependent fields	Dependent fields update only when the user changes the source field, not while the form is loading.	Pending
FT-08	onSubmit validation	Invalid data shows a clear message and the save is blocked; valid data is saved.	Pending
FT-09	Activate / deactivate rules	The behavior stops when a rule is inactive and returns when it is active; other rules are unaffected.	Pending
FT-10	No script errors	No JavaScript errors appear in the console for any rule.	Pending
PT-01	Form Load Time	Form loads in under 3 seconds.	Pending
PT-02	Field Change Response	Fields react immediately (under 1 second) without reloading the form.	Pending
PT-03	Many Rules on One Form	No slowdown, no conflicting rules and the same values always give the same behavior.	Pending

7. Results

7.1 Output Screenshots

The expected form behavior for each State is as follows.

State	Close code / Close notes	On hold reason	Editable fields	Message shown
New / In Progress	Optional	Hidden	Yes	Guidance on load
On Hold	Optional	Visible	Yes	Select the hold reason
Resolved	Mandatory	Hidden	Yes	Close fields required
Closed	Mandatory	Hidden	Read-only (controlled fields)	Incident is closed

The screenshot shows the ServiceNow Client Script editor interface. The title bar indicates the script is named "Client Script - Prevent state change via list edit". The main form contains the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onLoad
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒

The **Description** and **Messages** fields are empty. The **Script** field contains the following JavaScript code:

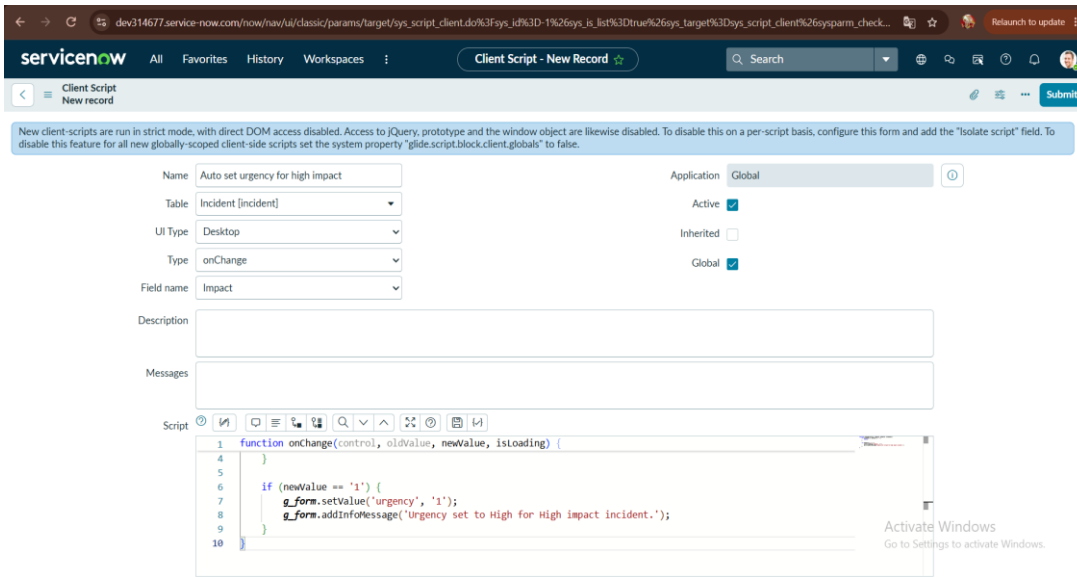
```
1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
2  
3     alert('State cannot be updated using list editing. Please open the Incident.');4  
5     callback(false);  
6 }  
7
```

The screenshot shows the ServiceNow Client Script editor interface for a new record. The title bar indicates the script is named "Client Script - New Record". The main form contains the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onCellEdit
- Field name:** State
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒

The **Description** and **Messages** fields are empty. The **Script** field contains the following JavaScript code:

```
1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
2  
3     alert('State cannot be updated using list editing. Please open the Incident.');4  
5     callback(false);  
6 }  
7
```



8. Advantages & Disadvantages

Advantages	Disadvantages
• The form reacts instantly, without submitting • Required data is captured consistently • Relevant fields only – a shorter, clearer form • UI Policies need little or no code • Patterns are reusable for other forms	• Rules run only in the browser form, not for API or import updates • Not a security control – ACLs are still needed • Conflicts are possible between policies and scripts • Scripts need maintenance when fields change • Too many rules can make the form harder to manage

9. Conclusion

The project shows how UI Policies and Client Scripts can control the ServiceNow Incident form dynamically. UI Policies make fields mandatory, read-only or visible when conditions are met, and onLoad, onChange and onSubmit Client Scripts add guidance, dependent-field updates and validation. Together they give users clear, immediate feedback and help produce complete and consistent Incident records. The test results from Section 6 should be added here once the tests have been executed.

10. Future Scope

- Add Data Policies or Business Rules to enforce the same rules for API and import updates.
- Apply the same patterns to other forms such as Problem and Change.
- Move the rules between instances with update sets.
- Automate verification using the Automated Test Framework.
- Extend the rules to Service Portal and mobile forms.

11. Appendix

Source Code

onChange Client Script – State (Table: incident):

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    // Do nothing while the form is loading or when the value is cleared
    if (isLoading || newValue === '') {
        return;
    }
    // Resolved (6) or Closed (7): remind the user about the closure fields
    if (newValue == '6' || newValue == '7') {
        g_form.showFieldMsg('close_notes', 'Close code and Close notes are required.', 'info');
    } else {
        g_form.hideFieldMsg('close_notes', true);
    }
    // On Hold (3): guide the user to the hold reason
    if (newValue == '3') {
        g_form.showFieldMsg('hold_reason', 'Select why the incident is on hold.', 'info');
    }
}
```

onSubmit Client Script (Table: incident):

```
function onSubmit() {
    var state = g_form.getValue('state');
    var shortDesc = g_form.getValue('short_description').trim();
    if (shortDesc === '') {
        g_form.showFieldMsg('short_description', 'Short description is required.', 'error');
        return false; // stop the save
    }
    if ((state == '6' || state == '7') && g_form.getValue('close_notes').trim() === '') {
        g_form.showFieldMsg('close_notes', 'Close notes are required to resolve or close.',
        'error');
        return false; // stop the save
    }
    return true; // data is valid, allow the save
}
```

Dataset Link

No external dataset was used. Sample Incident records were created manually with different States and Categories.

GitHub & Project Demo Link

GitHub: <https://github.com/<username>/<repository>>

Demo: <https://<your-demo-link>>

(replace with your own links)